

SQL Injection Detection Gap — Part 1:

Diagnosing a Web Layer Detection Gap in SQL Injection Coverage

A structured investigation I conducted into why a documented, active IDS signature did not fire on a successful SQL injection attack, tracing the issue through network configuration, payload encoding, and rule logic.

Overview

Following the successful validation of SSH brute force detection in Part 1, I tested a different attack category against the same lab: SQL injection against the Damn Vulnerable Web Application (DVWA), running on the victim VM. My goal was to confirm whether the detection stack (Suricata IDS feeding into Wazuh SIEM) could identify a web application layer attack the same way it identified the authentication layer attack.

The result was not a clean detection, it was a well isolated gap. This report documents my investigation as a case study in detection engineering: methodically ruling out causes one layer at a time, rather than treating a missed alert as a dead end.

Attack Summary

Using DVWA's SQL Injection module (set to Low security), I submitted two payload variants against the victim machine's User ID field, both successfully exploiting the vulnerability and returning all five user records, including password hashes, directly from the database.

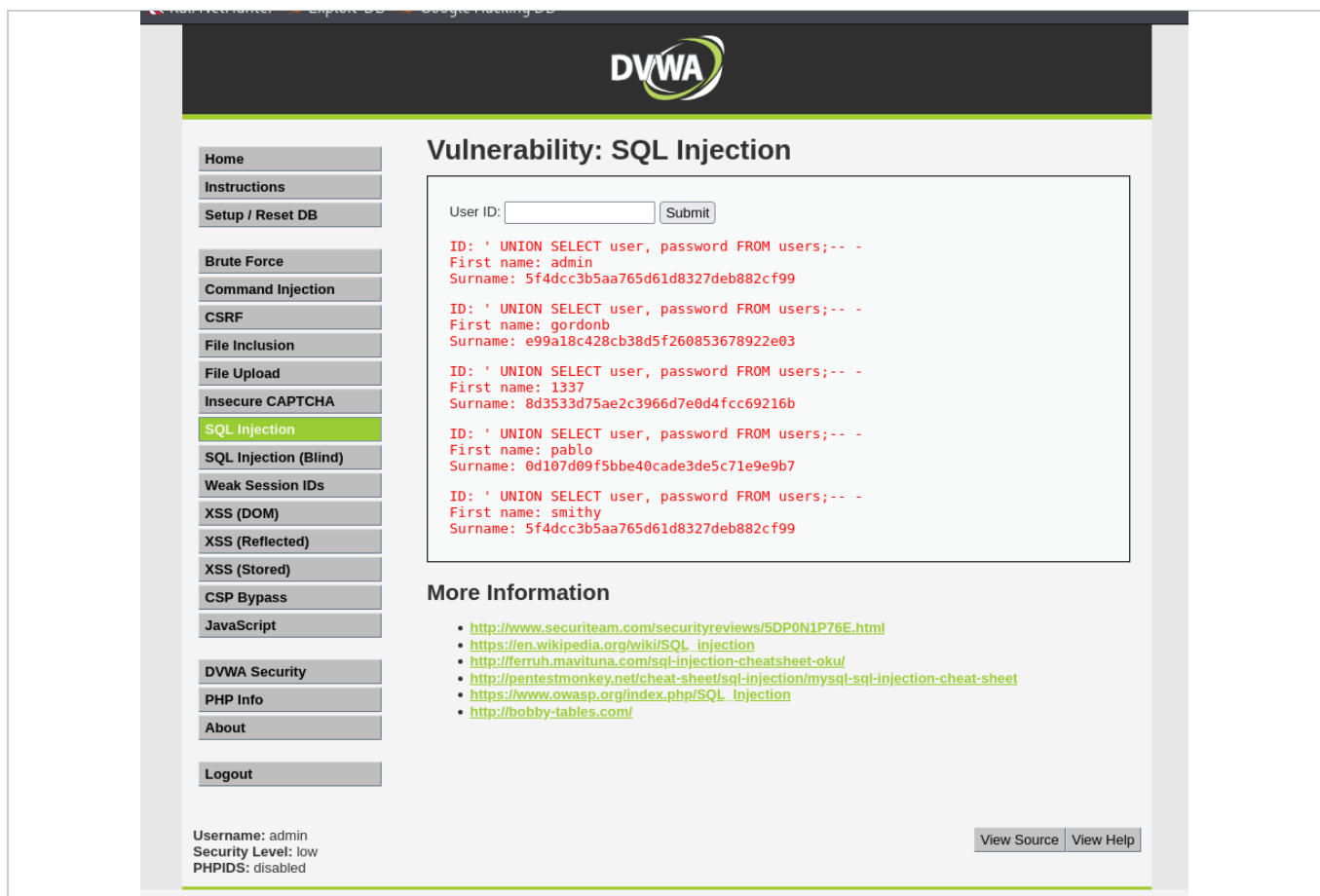


Figure 1: A UNION based SQL injection payload successfully dumps all user credentials from the DVWA database.

This confirmed the attack itself was fully successful at the application layer. The open question was whether the network based intrusion detection system watching this traffic would notice.

Investigation Timeline

Rather than treat the missing alert as a single failure, I proceeded with a structured elimination process: each step tested one specific hypothesis before moving to the next.

Step	Hypothesis Tested	Result
1	Is Suricata seeing HTTP traffic on port 8080 at all?	No, HTTP_PORTS was scoped to port 80 only
2	Fix HTTP_PORTS to include 8080, does traffic get inspected now?	Yes, confirmed via raw HTTP event logs
3	Does a basic ' OR '1'='1 payload trigger any alert?	No alert fired
4	Does a noisier UNION SELECT payload trigger any alert?	No alert fired
5	Does a matching rule exist for this attack pattern at all?	Yes, rule 2006446 exists and is active
6	Does the payload satisfy the rule's exact match pattern?	Yes, confirmed byte for byte in the captured URL
7	Does Wazuh show anything independently of Suricata?	No, Wazuh only relays what Suricata flags

Finding 1: A Real Configuration Bug

My first hypothesis, and the one that turned out to be genuinely wrong in the setup, was that Suricata might not be inspecting traffic on DVWA's port at all. DVWA runs on port 8080, but Suricata's configuration defined its HTTP_PORTS variable as only port 80. Since IDS rules that match on HTTP content are scoped to this variable, any rule requiring HTTP layer inspection would silently skip all traffic on 8080, even though Suricata was still logging the raw connection as HTTP traffic, which made the gap easy to miss at first glance.

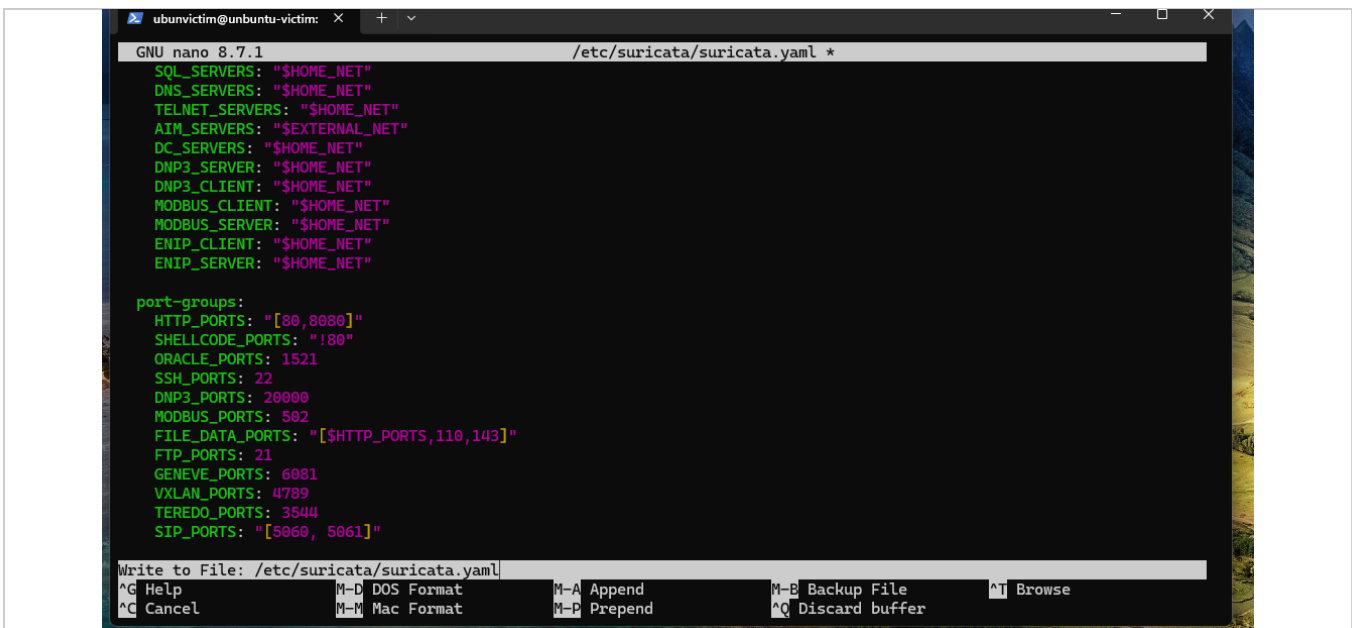


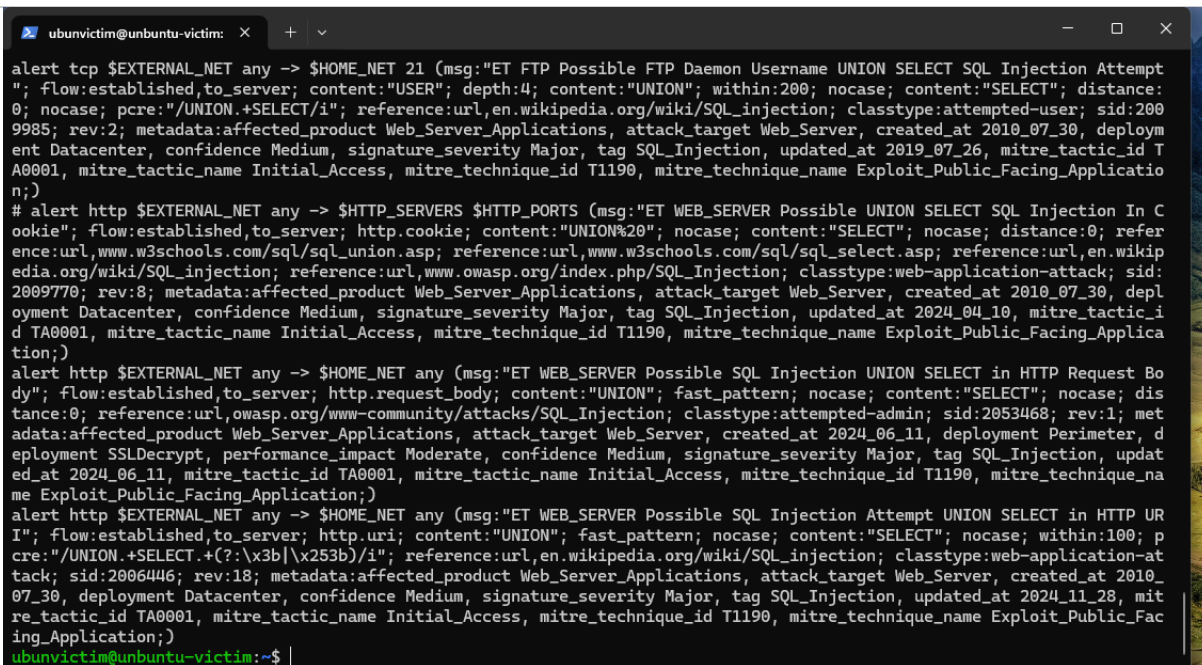
Figure 2: Correcting Suricata's HTTP_PORTS variable from "80" to "[80,8080]" so rule based inspection applies to DVWA's port.

After I applied this fix and reloaded Suricata's full ruleset (52,679 signatures), the traffic visibility issue was resolved and confirmed: Suricata was now actively inspecting requests to port 8080, verified against a fresh HTTP request log.

Finding 2: A Rule That Should Have Fired, But Didn't

With traffic visibility confirmed, I re tested both injection payloads. Neither produced a Suricata alert. A search of the loaded ruleset confirmed that a relevant, actively loaded signature already existed for this exact attack pattern:

Rule Element	Value
Signature ID	2006446
Message	ET WEB_SERVER Possible SQL Injection Attempt UNION SELECT in HTTP URI
Match logic	Requires "UNION", then "SELECT", then a literal or encoded semicolon (; or %3b)
MITRE Technique	T1190, Exploit Public Facing Application
Payload tested	' UNION SELECT user, password FROM users;-- -
Payload outcome	Successfully dumped all 5 user credential hashes at the application layer
Rule outcome	Did not fire, despite the payload satisfying the rule's documented pattern



```

alert tcp $EXTERNAL_NET any -> $HOME_NET 21 (msg:"ET FTP Possible FTP Daemon Username UNION SELECT SQL Injection Attempt"; flow:established,to_server; content:"USER"; depth:4; content:"UNION"; within:200; nocase; content:"SELECT"; distance:0; nocase; pcre:"/UNION.+SELECT/i"; reference:url,en.wikipedia.org/wiki/SQL_injection; classtype:attempted-user; sid:2009985; rev:2; metadata:affected_product Web_Server_Applications, attack_target Web_Server, created_at 2010_07_30, deployment Datacenter, confidence Medium, signature_severity Major, tag SQL_Injection, updated_at 2019_07_26, mitre_tactic_id T1190, mitre_tactic_name Initial_Access, mitre_technique_id T1190, mitre_technique_name Exploit_Public_Facing_Application;)

# alert http $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (msg:"ET WEB_SERVER Possible UNION SELECT SQL Injection In Cookie"; flow:established,to_server; http.cookie; content:"UNION%20"; nocase; content:"SELECT"; nocase; distance:0; reference:url,www.w3schools.com/sql/sql_union.asp; reference:url,www.w3schools.com/sql/sql_select.asp; reference:url,en.wikipedia.org/wiki/SQL_injection; reference:url,www.owasp.org/index.php/SQL_Injection; classtype:web-application-attack; sid:2009770; rev:8; metadata:affected_product Web_Server_Applications, attack_target Web_Server, created_at 2010_07_30, deployment Datacenter, confidence Medium, signature_severity Major, tag SQL_Injection, updated_at 2024_04_10, mitre_tactic_id T1190, mitre_tactic_name Initial_Access, mitre_technique_id T1190, mitre_technique_name Exploit_Public_Facing_Application;)

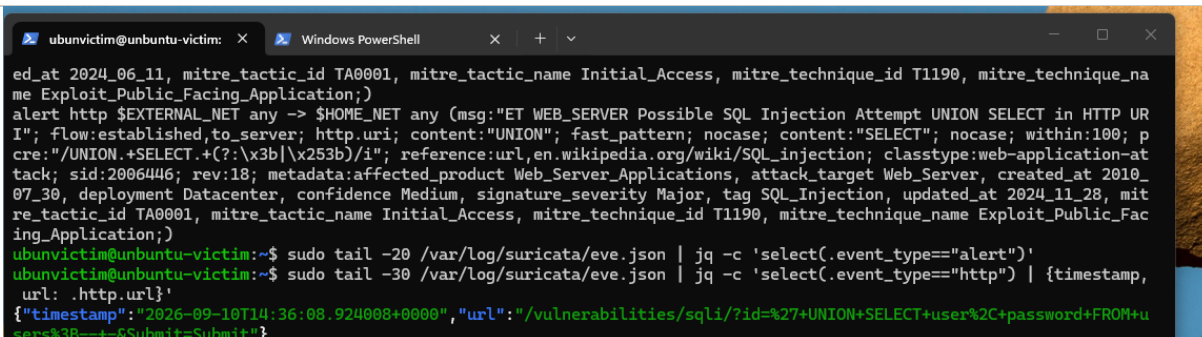
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"ET WEB_SERVER Possible SQL Injection UNION SELECT in HTTP Request Body"; flow:established,to_server; http.request_body; content:"UNION"; fast_pattern; nocase; content:"SELECT"; nocase; distance:0; reference:url,owasp.org/www-community/attacks/SQL_Injection; classtype:attempted-admin; sid:2053468; rev:1; metadata:affected_product Web_Server_Applications, attack_target Web_Server, created_at 2024_06_11, deployment Perimeter, deployment SSLDecrypt, performance_impact Moderate, confidence Medium, signature_severity Major, tag SQL_Injection, updated_at 2024_06_11, mitre_tactic_id T1190, mitre_tactic_name Initial_Access, mitre_technique_id T1190, mitre_technique_name Exploit_Public_Facing_Application;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"ET WEB_SERVER Possible SQL Injection Attempt UNION SELECT in HTTP URI"; flow:established,to_server; http.uri; content:"UNION"; fast_pattern; nocase; content:"SELECT"; nocase; within:100; pcre:"/UNION.+SELECT.+(?:\x3b|\x253b)/i"; reference:url,en.wikipedia.org/wiki/SQL_injection; classtype:web-application-attack; sid:2006446; rev:18; metadata:affected_product Web_Server_Applications, attack_target Web_Server, created_at 2010_07_30, deployment Datacenter, confidence Medium, signature_severity Major, tag SQL_Injection, updated_at 2024_11_28, mitre_tactic_id T1190, mitre_tactic_name Initial_Access, mitre_technique_id T1190, mitre_technique_name Exploit_Public_Facing_Application;)
ubunvictim@ubuntu-victim:~$

```

Figure 3: Four SQL injection related signatures found in the loaded ruleset, including rule 2006446, the closest match for this attack.

The rule's own pattern condition (UNION, then SELECT, then a semicolon) was deliberately satisfied in the final test payload, and the resulting HTTP request log confirms the encoded semicolon (%3B) was present exactly where the rule's pattern expects it:



```

ed_at 2024_06_11, mitre_tactic_id T1190, mitre_tactic_name Initial_Access, mitre_technique_id T1190, mitre_technique_name Exploit_Public_Facing_Application;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"ET WEB_SERVER Possible SQL Injection Attempt UNION SELECT in HTTP URI"; flow:established,to_server; http.uri; content:"UNION"; fast_pattern; nocase; content:"SELECT"; nocase; within:100; pcre:"/UNION.+SELECT.+(?:\x3b|\x253b)/i"; reference:url,en.wikipedia.org/wiki/SQL_injection; classtype:web-application-attack; sid:2006446; rev:18; metadata:affected_product Web_Server_Applications, attack_target Web_Server, created_at 2010_07_30, deployment Datacenter, confidence Medium, signature_severity Major, tag SQL_Injection, updated_at 2024_11_28, mitre_tactic_id T1190, mitre_tactic_name Initial_Access, mitre_technique_id T1190, mitre_technique_name Exploit_Public_Facing_Application;)

ubunvictim@ubuntu-victim:~$ sudo tail -20 /var/log/suricata/eve.json | jq -c 'select(.event_type=="alert")'
ubunvictim@ubuntu-victim:~$ sudo tail -30 /var/log/suricata/eve.json | jq -c 'select(.event_type=="http") | {timestamp, url: .http.url}'
{"timestamp": "2026-09-10T14:36:08.924008+0000", "url": "/vulnerabilities/sqli/?id=%27+UNION+SELECT+user%2C+password+FROM+users%3B--+&Submit=Submit"}

```

Figure 4: The alert check returns no results, while the HTTP log directly below confirms the exact payload, semicolon included, was captured by Suricata.

Unresolved finding

The signature is loaded, active, and its documented match conditions were satisfied by the captured traffic, yet no alert was generated. This points to a deeper Suricata engine level behavior (possibly related to stream reassembly or flow state matching for this specific traffic pattern) that I was not able to resolve within this session and have flagged for further investigation.

Finding 3: Wazuh Correctly Reflected the Upstream Gap

My check of the Wazuh SIEM dashboard for the same time window showed no corresponding alert either. This is expected, not a separate failure: Wazuh's web attack detection in this architecture is driven by ingesting Suricata's own alert log, rather than performing independent packet level analysis. Since Suricata never generated an alert type event for either injection attempt, there was nothing for Wazuh to relay, confirming the gap traces to a single root cause rather than two independent failures.

Why This Finding Matters

A clean, single detection win is a good demonstration of a working pipeline. A well-documented gap, traced methodically through configuration, traffic capture, and rule logic, demonstrates something more valuable in a security operations context: the ability to investigate why a control failed, rather than only confirming when it succeeds. Real SOC and detection engineering work involves exactly this kind of layered elimination process.

- Diagnosed and permanently fixed a real Suricata configuration bug (HTTP_PORTS scope) that was silently blocking web layer detection on a non-standard port.
- Distinguished between “traffic visibility” and “rule matching” as two separate, independently verifiable layers of detection.
- Located the specific signature relevant to the attack technique and confirmed its match conditions against captured traffic at the byte level.
- Correctly reasoned through Wazuh's dependency on Suricata's alert output, avoiding a false conclusion of “two separate broken tools.”
- Produced a reproducible, well scoped open question for further investigation, rather than an unexplained dead end.